

FastAPI'de Kalıcılık: SQLAlchemy ve CRUD Operasyonları

NTP II – Hafta 10



FastAPI'de Kalıcılık: SQLAlchemy ve CRUD Operasyonları

Bu hafta, API uygulamalarımızda verilerin kalıcılığını (persistence) nasıl sağlayacağımızı öğreneceğiz.

Geçici bellek içi depolamanın (in-memory) yetersiz kaldığı durumları inceleyeceğiz.

Object-Relational Mapper (ORM) konseptini ve Python'daki en güçlü ORM olan SQLAlchemy'yi tanıyacağız.



Modül Öğrenme Hedefleri

Üretim (production) ortamları için 'in-memory' depolamanın neden uygun olmadığını anlamak (veri kaybı, ölçeklenememe).

ORM (Object-Relational Mapper) kavramını ve SQLAlchemy'nin Python objelerini SQL'e çevirmedeki rolünü öğrenmek.

SQLAlchemy Base, Engine ve Session kavramlarını kullanarak veritabanı bağlantısı kurmak (SQLite ile).

Öğrenme Hedefleri 2

OOP'de İkinci Aşama: Pydantic modeline ek olarak bir SQLAlchemy ORM modeli tanımlamak.

FastAPI'nin 'Dependency Injection' (Bağımlılık Enjeksiyonu) sistemini kullanarak her istek için veritabanı oturumunu yönetmek.

Tüm CRUD (Create, Read, Update, Delete) operasyonlarını kalıcı olarak veritabanı üzerinde gerçekleştirmek.

Müşteri Talebi: Veri Kaybı Felaketi

Senaryo: 'Sunucuyu yeniden başlattık (uicorn'u kapattık) ve analistlerin girdiği tüm IoC'ler (Indicators of Compromise) kayboldu.'
Geçici (volatile) liste depolama, siber güvenlik gibi kritik uygulamalar için kabul edilemezdir.
Eklenen tüm IoC'lerin kalıcı (persistent) olması talep edilmektedir.

Gereksinim 1: SQLite Entegrasyonu

Projemize hızlı ve kolay entegrasyon için bir SQLite veritabanı eklenmesi istenmektedir.

SQLite, sunucusuz yapısıyla geliştirme ortamlarında hızlı prototipleme imkanı sunar.

Üretim ölçeği için daha sonra PostgreSQL veya MySQL gibi sunucu tabanlı sistemlere geçiş yapılabilir.



Gereksinim 2: POST İşlemi

Tüm POST /indicators API isteklerinin, bellek içi bir listeye değil, kalıcı olarak bu veritabanına yazılması zorunludur.
Bu, veri ekleme (Create) operasyonunu kalıcı hale getirir.



Gereksinim 3: GET İşlemi

GET /indicators endpoint'inin artık verileri listeden değil, eklenen bu SQLite veritabanından okuması gerekir.
Bu, uygulamanın yeniden başlatılmasından sonra bile doğru ve güncel veriye erişimi garantiler.



Gereksinim 4: Tekil CRUD Operasyonları

Tek bir göstergeyi ID ile getirecek (GET /indicators/{id}) endpoint'ine ihtiyaç vardır.

Mevcut göstergeyi güncelleyecek (PUT /indicators/{id}) endpoint'i gereklidir.
Belirli bir göstergeyi silecek (DELETE /indicators/{id}) endpoint'i gereklidir.



ORM Kavramına Derin Bakış

ORM'ler, geliştiricilerin veritabanı işlemlerini programlama dillerindeki nesneler (objeler) üzerinden yapmasına olanak tanır.

SQL sorgularını manuel yazma yükünü azaltır ve daha güvenli, okunabilir kod sağlar.

SQLAlchemy, SQL dilini bilmeden Python sınıfları ile tabloları yönetmemizi sağlar.

SQLAlchemy'nin Rolü

SQLAlchemy, Python objelerindeki değişiklikleri otomatik olarak SQL INSERT, UPDATE veya DELETE komutlarına çevirir.
Aynı zamanda, veritabanından dönen satırları (rows) tekrar anlamlı Python objelerine dönüştürür.
Bu süreç, 'eşleme' (mapping) olarak adlandırılır.

Pedagojik Odak: İki Model Paradigması

Bu hafta en kritik pedagojik ayırım, neden projede iki farklı 'class' (sınıf) tanımladığımızdır.

Öğrenci sorusu: Biri yeterli değil miydi?

Cevap, her sınıfın farklı bir sorumluluk alanını temsil etmesinde yatar.



Pydantic Modeli (IoCSchema)

Pydantic, API'nin dış dünya ile (istemcilerle) JSON formatında konuşma şeklidir.

Temel görevi gelen veriyi (payload) doğrulamaktır (veri doğrulama, tip kontrolü, zorunlu alanlar).

FastAPI, Pydantic'i kullanarak otomatik JSON şeması belgeleri (Swagger/OpenAPI) üretir.



SQLAlchemy Modeli (Indicator)

SQLAlchemy modeli, API'nin veritabanı ile (SQL) konuşma şeklidir. Temel görevi, Python sınıfını veritabanındaki belirli bir tabloya eşlemektir (Persistence). Bu model, veritabanı alanlarını, tiplerini ve kısıtlamalarını tanımlar.

OOP Perspektifi

Her iki modelin de 'class' olması, Nesne Yönelimli Programlamanın (OOP) soyutlama gücüdür.

Bu tasarım, Tek Sorumluluk Prensibine (Single Responsibility Principle) uyar:

Schema Class (Pydantic): Doğrulama ve Sunum.

ORM Class (SQLAlchemy): Kalıcılık ve Veri Erişim Katmanı (DAL).

Yeni Bağımlılıkların Kurulumu

Projemizin artık harici bir kütüphaneye ihtiyacı var: SQLAlchemy.
Kurulum komutu: 'pip install sqlalchemy'.
SQLite kullandığımız için ek bir veritabanı sürücüsü (driver) kurmaya gerek yoktur.



Kodun yönetilebilirliği ve bakımı için projeyi şimdiden modüler dosyalara ayırmak en iyi pratiktir.

Ayracağımız ana dosyalar:

1. main.py (API endpoint'leri)
2. database.py (SQLAlchemy kurulumu)
3. models.py (SQLAlchemy modelleri)
4. schemas.py (Pydantic modelleri)

'database.py' Oluşturuluyor

Veritabanı bağlantısı ve oturum yönetimi için gerekli sınıfları import ediyoruz.

```
'from sqlalchemy import create_engine'
```

```
'from sqlalchemy.ext.declarative import declarative_base'
```

```
'from sqlalchemy.orm import sessionmaker'
```

Veritabanı URL'si Tanımı

Veritabanına bağlanmak için kullanılan dize (string) tanımlanır.

```
'SQLALCHEMY_DATABASE_URL = 'sqlite:///./threat_intel.db''
```

Bu URL, veritabanı dosyasının projenin ana dizininde oluşacağını belirtir (threat_intel.db).



Engine (Motor) Oluşturma

'engine = create_engine(...)' komutu, veritabanına bağlanmak için ana arabirimi oluşturur.

Engine, veritabanının hangi diyalekti (SQLite, Postgres vb.) kullandığını bilir ve bağlantı havuzunu yönetir.

SQLite'a Özel Bağlantı Ayarı

FastAPI asenkron çalıştığı için (birden fazla iş parçacığı), SQLite'in varsayılan kısıtlamasını aşmalıyız.

'connect_args={'check_same_thread': False}' satırı bu amaçla eklenir.

Bu ayar, sadece SQLite kullanılırken zorunludur.

Oturum Fabrikası: 'SessionLocal'

'SessionLocal = sessionmaker(...)' oturum nesneleri oluşturmak için kullanılan bir fabrika sınıfıdır.
Her veritabanı işlemi (istek) için yeni bir oturum buradan oluşturulacaktır.

Oturum Ayarları

'autocommit=False': Otomatik COMMIT yapma. Biz manuel olarak

'db.commit()' çağıracağız.

'autoflush=False': Sorgular çalıştırılmadan nesneleri VT'ye senkronize etme.

'bind=engine': Bu oturumların hangi motorla bağlantı kuracağını belirtir.

ORM Temel Sınıfı: 'Base'

'Base = declarative_base()' ifadesi, ORM eşlemesini sağlayan temel sınıfı oluşturur.
Tüm SQLAlchemy modellerimiz (örneğin Indicator) bu Base sınıfından miras almak zorundadır.
Bu Base, veritabanı meta verilerini tutar.

'models.py' Oluşturuluyor

Bu dosya, veritabanındaki tabloları temsil eden Python sınıflarını içerecektir.

```
'from sqlalchemy import Column, Integer, String'
```

```
'from .database import Base'
```

Unutmayın: Bu, hafta 2'deki Pydantic modelinden farklıdır.

Indicator Sınıfı: Tablo Eşlemesi

'class Indicator(Base):' Base sınıfından miras alıyoruz.
'__tablename__' = 'indicators' bu sınıfın veritabanındaki 'indicators' tablosuna karşılık geldiğini belirtir.

Indicator Sütun Tanımları

'id = Column(Integer, primary_key=True, index=True)': Birincil anahtar ve integer tipinde, sorgulama için indekslenmiş.

'value = Column(String, index=True)': Göstergenin içeriği (örn: IP adresi, dosya hash'i).

'type = Column(String)': Göstergenin tipi.

Indicator Sütun Tanımları 2

'threat_level = Column(Integer, default=3)': Tehdit seviyesi integer olarak tutulur.

'default=3' ayarı sayesinde, kullanıcı bu alanı belirtmezse varsayılan değer 3 olarak atanır.



'main.py' Güncellemesi: Tablo Oluşturma

Uygulama başladığında, veritabanı tablolarının gerçekten oluşturulduğundan emin olmalıyız.

'models.Base.metadata.create_all(bind=engine)' komutu bunu sağlar. Bu komut, tanımlanan ORM modellerine göre SQLite dosyasında tabloları oluşturur.

Bağımlılık Enjeksiyonu (Dependency Injection)

FastAPI'nin DI sistemi (Depends), veritabanı oturumu gibi kaynakları yönetmek için kullanılır.

Bu sistem, her API isteği için yeni bir oturum açıp, istek bitince (hatalı olsa bile) kapatmayı garanti eder.



'get_db' Bağımlılık Fonksiyonu

Bu fonksiyon, 'SessionLocal()' kullanarak bir veritabanı oturumu başlatır. Oluşturulan oturum ('db'), 'try' bloğu içinde FastAPI rotalarına iletilir. Oturum nesnesi 'sqlalchemy.orm.Session' tipindedir.



'yield' ve 'finally' Kullanımı

'yield db' ifadesi, oturumu API rotasına teslim eder ve işlemin tamamlanmasını bekler.

İstek tamamlandığında (veya bir hata oluştuğunda), kontrol 'finally' bloğuna geçer.

'finally' bloğundaki 'db.close()' komutu, oturumun güvenli bir şekilde kapatılmasını sağlar ve kaynak sızıntısını önler.

CREATE Operasyonu Tanımı (POST)

'@app.post('/indicators', response_model=schemas.LoCSchema)' rotası tanımlanır.

Fonksiyon parametreleri:

1. 'ioc: schemas.LoCCreateSchema' (Gelen JSON verisi, Pydantic ile doğrulanır).
2. 'db: Session = Depends(get_db)' (Veritabanı oturumu).

Model Dönüşümü

Doğrulanmış Pydantic objesi, veritabanına yazılabilmesi için SQLAlchemy modeline çevrilmelidir.

```
'db_indicator = models.Indicator(**ioc.model_dump())'
```

'model_dump()' metodu Pydantic verisini anahtar-değer çiftleri olarak SQLAlchemy modeline aktarır.



Veritabanına Yazma Aşaması 1: Ekleme

'db.add(db_indicator)' komutu, yeni Indicator nesnesini geçerli veritabanı oturumuna ekler.

Bu aşamada veri henüz kalıcı olarak veritabanı dosyasına yazılmamıştır; sadece oturumun belleğinde tutulur.



Veritabanına Yazma Aşaması 2: Kaydetme

'db.commit()' komutu, oturumda biriken tüm değişiklikleri (transaction) kalıcı olarak veritabanına yazar.
Bu noktada SQL INSERT komutu çalıştırılmış olur.



Veritabanına Yazma Aşaması 3: Yenileme

'db.refresh(db_indicator)' komutu, veritabanının atadığı otomatik değerleri (örneğin ID) Python objesine geri yükler. Böylece API yanıtı (response_model) doğru ID ile oluşturulabilir.

READ Operasyonu: Tümünü Getirme

'@app.get('/indicators', response_model=List[schemas.loCSchema])' rotası tanımlanır.

Sayfalama (Pagination) için 'skip' ve 'limit' sorgu parametreleri kullanılır.



Sorgu Oluşturma ve Sayfalama

'db.query(models.Indicator)': Indicator tablosunu sorgulamaya başla.
'offset(skip)': Belirtilen sayı kadar kaydı atla (sayfalama başlangıcı).
'limit(limit)': Maksimum döndürülecek kayıt sayısını belirle.
'all()': Tüm sonuçları Python listesi olarak getir.

READ Operasyonu: Tek Kayıt Getirme

Belirli bir göstergelyi ID üzerinden getirmek için endpoint tanımlanır. Fonksiyon parametresi olarak 'indicator_id: int' kullanılır.

Filtreleme ve İlk Kaydı Alma

```
'db.query(models.Indicator)'  
'filter(models.Indicator.id == indicator_id)': Sorguyu belirli bir ID ile filtrele.  
'first()': Filtrelenen sonuçlardan yalnızca ilkinin al (ID benzersiz olduğu için bir  
veya sıfır sonuç döner).
```

Hata Yönetimi: 404 (Not Found)

Profesyonel API tasarımında, kullanıcı bir kaynağa erişmek istediğinde ve kaynak mevcut değilse 404 HTTP durum kodu döndürülmelidir. Eğer 'db_indicator' nesnesi 'None' ise (yani kayıt bulunamazsa), hata fırlatılır.

FastAPI'de Hata Fırlatma

FastAPI, standart HTTP hatalarını fırlatmak için 'HTTPException' sınıfını sağlar. 'raise HTTPException(status_code=404, detail='Indicator not found')' komutu kullanılır.

FastAPI bu hatayı otomatik olarak standart bir JSON yanıtına dönüştürür.

UPDATE Operasyonu Hazırlığı (PUT)

Güncelleme, CREATE ve READ işlemlerinin bir kombinasyonudur.

1. Öncelikle güncellenecek kaydı ID ile veritabanından oku (ve 404 kontrolü yap).
2. Gelen Pydantic verilerini ORM objesi üzerine kopyala/ata.

Gerçekleştirme: UPDATE Süreci

Örnek veri ataması: `'db_indicator.value = ioc.value'`.

Bu atamadan sonra SQLAlchemy oturumu, objenin değiştiğini otomatik olarak takip eder.

Değişiklikleri kalıcı hale getirmek için yine `'db.commit()'` ve `'db.refresh()'` çağrılmalıdır.

DELETE Operasyonu (DELETE)

Silme operasyonu için de hedef kaydın ID ile bulunması gereklidir (yine 404 kontrolü yapılır).
Kayıt bulunduktan sonra, silme işlemi 'db.delete()' metodu ile gerçekleştirilir.



Gerçekleştirme: DELETE Süreci

1. 'db_indicator = read_indicator(indicator_id, db)' (Kaydı bulma)
2. 'db.delete(db_indicator)': Nesneyi oturumdan silmek üzere işaretle.
3. 'db.commit()': Değişikliği veritabanına kalıcı olarak uygula.

SQLAlchemy ORM vs. Ham SQL

Bu hafta, ham SQL sorguları yazmak yerine ORM kullanmanın avantajlarını tartışmak için iyi bir fırsattır.

ORM, genellikle geliştirme hızını artırır ve kodun veritabanı türünden bağımsızlığını destekler.

ORM'nin Güvenlik Katkısı

SQLAlchemy gibi ORM'ler, doğru kullanıldığında SQL Enjeksiyon saldırılarını otomatik olarak zorlaştırır.

ORM'ler, kullanıcı tarafından sağlanan verileri (parametreleri), SQL sorgu komutundan ayırır.

Bu, verinin kötü amaçlı komut olarak çalışmasını engeller.



Öğretmenler için not: 'connect_args={'check_same_thread': False}' satırı yalnızca SQLite için gereklidir.
Proje PostgreSQL, MySQL veya MariaDB gibi farklı bir veritabanına geçtiğinde bu argüman mutlaka kaldırılmalıdır.



Bir API'nin başarısını gösteren, sadece başarılı yanıtlar (200, 201) değil, aynı zamanda doğru hata yanıtlarıdır (4xx, 5xx).
404 hatasını manuel olarak 'HttpException' ile fırlatmak, profesyonel API tasarımının temel bir gereğidir.



FastAPI'de Tip İpuçları (Type Hinting)

Tüm fonksiyon tanımlarında kullanılan 'List[schemas.ioCSchema]' ve 'db: Session' gibi tip ipuçları (type hints) kritiktir. Bunlar kodun anlaşılmasını ve editörlerin otomatik tamamlama özelliklerini kullanmasını sağlar.

Veri Şeması Ayırımının Detayları

CREATE operasyonu için genellikle 'loCCreateSchema' kullanılır (ID alanı hariç tutulabilir).

READ operasyonu için ise 'loCSchema' kullanılır (ID dahil tüm alanları içerir).
Bu ayırım, API güvenliğini ve esnekliğini artırır.

CRUD Model Metotları (Repository Pattern)

Büyük projelerde, 'db.add()', 'db.query()' gibi veritabanı işlemlerini doğrudan 'main.py' de yapmak yerine, ayrı bir 'servis' veya 'repository' katmanına taşınması önerilir.

Bu, bağımlılıkları daha da soyutlar ve iş mantığını API rotalarından ayırır.



FastAPI ve SQLAlchemy 2.0

Kaynakta gösterilen yöntem senkronizedir (FastAPI tarafından thread pool'da çalıştırılır).

SQLAlchemy'nin yeni sürümleri, tam asenkron (async/await uyumlu) veritabanı sürücülerini destekler.

Bu, I/O operasyonlarında daha yüksek paralelizm ve performans sağlar.



Veritabanı Şeması Geçişleri (Migrations)

Uygulama başladığında 'create_all(bind=engine)' ile tablo oluşturmak yalnızca geliştirme aşaması için uygundur.
Üretim ortamında, tabloda yapılan değişiklikleri (yeni sütunlar, silinen alanlar) yönetmek için ****Alembic**** gibi araçlar kullanılır.

SQLAlchemy ilişkileri (Relationships)

Şu anki 'Indicator' tablomuz bağımsızdır.
Gerçek dünyada, tablolar arasında ilişkiler (örneğin: Bir tehdit aktörünün birden fazla göstergesi vardır) kurmamız gerekir.
SQLAlchemy, bu ilişkileri (One-to-Many, Many-to-Many) Python seviyesinde yönetmemizi sağlar.

İndekslemenin Önemi

Indicator modelimizde 'id' ve 'value' alanlarına 'index=True' ekledik. İndeksler, veritabanının sorgu sürelerini (özellikle 'WHERE' veya 'ORDER BY' kullanılan sorgularda) dramatik şekilde hızlandırır. Ancak, yazma (INSERT/UPDATE) işlemlerini yavaşlatabilir; bu bir denge gerektirir.



Özet: Kalıcılık Neden Başarılı Oldu?

Uygulama artık verileri geçici bellekte değil, kalıcı bir dosya sistemi tabanlı veritabanında (SQLite) tutuyor.
Sunucu (uvicorn) yeniden başlatılsa bile, 'threat_intel.db' dosyası korunur ve veriler kalıcı olur.



Özet: SQLAlchemy Temel Yapıları

Engine: Veritabanı bağlantısını kuran motor.

Base: ORM modellerinin miras aldığı temel sınıf.

Session: Veritabanı ile konuşma ve işlem (transaction) birimi.



Özet: Model Ayrımı

Pydantic (schemas.py): API girişi/çıkışı ve doğrulama.
SQLAlchemy (models.py): Veritabanı tablo eşlemesi ve kalıcılık.

Özet: Oturum Yönetimi ve DI

'get_db' fonksiyonu ve 'yield' kullanımı, kaynakların güvenli açılmasını ve kapanmasını garantiledi.
FastAPI'nin 'Depends' sistemi, oturum bağımlılığını otomatik olarak tüm CRUD rotalarına enjekte etti.



Özet: CRUD Implementasyonu

POST ile yeni kayıt ekleme (CREATE) başarıyla gerçekleştirildi.
GET ile liste (READ All) ve ID bazlı (READ One) sorgulama sağlandı.
PUT ve DELETE operasyonları için temel yapılar kuruldu ve 404 hata yönetimi eklendi.



Siber Güvenlikte Kalıcılığın Önemi

IoC (Indicators of Compromise) verileri hızla değişir ve sürekli olarak erişilebilir olmalıdır.

Veri kalıcılığı, geçmiş tehditlerin analiz edilmesini ve gelecekteki saldırıların önlenmesini sağlar.

FastAPI ve SQLAlchemy, bu tür görev açısından kritik sistemler için hızlı ve güvenilir bir çözüm sunar.



Test Edilebilirlik

Bağımlılık Enjeksiyonu sayesinde, API rotalarımızı test ederken gerçek bir veritabanı bağlantısı yerine bir 'sahte' (mock) veritabanı oturumu enjekte edebiliriz.

Bu, birim testlerinin (unit test) daha hızlı ve güvenilir çalışmasını sağlar.



SQLAlchemy 2.0 Asenkron özelliklerini inceleyin.
Alembic ile veritabanı şeması geçişlerini (migrations) öğrenin.
FastAPI ve ORM'yi kullanarak karmaşık ilişkisel veri modelleri kurmayı deneyin.

Sorular ve Tartışma

Bu modül, bir API'nin sadece arayüz (interface) değil, aynı zamanda veriyi güvenli ve kalıcı yöneten bir sistem olduğunu gösterdi.
Hangi bölümlerin daha fazla açıklanmaya ihtiyacı var? Hangi teknik zorluklarla karşılaştınız?



FastAPI'de Kalıcılık: SQLAlchemy ve CRUD Operasyonları

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

